Local setup

Locally FDDT runs as: **Postgres**, **Redis** and **PgBouncer** (in Docker), the **FastAPI** app, the **Celery workers** (one per queue, or one that serves all three), **Celery beat**, and the **Vite dev server**. Examples are Windows/PowerShell first (the project is developed on Windows); bash equivalents are in comments. For production, see production-deployment.md.

Prerequisites

Tool	Version	Notes
Docker Desktop	any recent	Postgres 16, Redis 7, PgBouncer 1.23
Python	3.11+ (developed and tested on **3.14**; the Docker image uses 3.12)	
Node.js	20+ (developed on **24**)	
Azure resources	—	Storage account, Document Intelligence, Azure OpenAI (vision-capable deployment). Needed for the *pipeline*, not to start the app — see environment-variables.md

1. Start Postgres, Redis and PgBouncer

From the repo root:

```
docker compose up -d        # postgres, redis, pgbouncer
docker compose ps           # postgres and redis "healthy", pgbouncer "running"
```

This creates `docauth-postgres` (port 5432), `docauth-redis` (6379) and `docauth-pgbouncer` (6432) with named volumes (`postgres_data` , `redis_data`). The owner user/password/database are all `docauth` . Data persists across restarts; `docker compose down -v` deletes it. PgBouncer is optional locally (see step 2).

2. Backend

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1          # bash: source .venv/bin/activate
pip install -r requirements.txt       # several minutes (OpenCV, PyMuPDF, pikepdf, Azure SDKs)
copy .env.example .env                # bash: cp .env.example .env
```

Edit `backend/.env` : set `JWT_SECRET_KEY` and the Azure values (storage, Document Intelligence, OpenAI, and the Azure OpenAI rate caps at ~80 % of your deployment's quota). The Postgres/Redis defaults already match `docker-compose.yml` . To route the app through PgBouncer as production does, also set `DATABASE_POOLER_HOST=localhost` (port 6432); leave it unset to connect directly. **Always run backend commands from `backend/`** — the `.env` path is relative to the working directory.

Create the schema, the database roles and the first account:

```
alembic upgrade head        # 14 migrations → 19 tables, RLS policies, the app roles fddt_app /
                            # fddt_platform (passwords from DATABASE_APP_PASSWORD /
DATABASE_PLATFORM_PASSWORD),
                            # the "Default Company" with 38 rules, thresholds 30/60 and a few FAKE test
                            # issuers, and the platform rule template
python seed.py              # creates the PLATFORM ADMIN admin@example.com / ChangeMe123! (idempotent;
                            # re-running resets that password — set SEED_ADMIN_EMAIL / SEED_ADMIN_PASSWORD)
```

The migrations run as the **owner** (`DATABASE_URL`), which needs CREATEROLE but not superuser. The app itself connects as `fddt_app` / `fddt_platform` , never as the owner.

The seeded account is a **platform admin**: it belongs to no company and cannot submit or review cases. After logging in, under **Platform**: create a company (or use "Default Company"), then under **Platform › Users** create that company's `user` , `reviewer_l1` and `reviewer_l2` accounts. Sign in as those to exercise the case workflow.

3. Run the API

```
uvicorn app.main:app --reload        # http://localhost:8000
```

Check: `http://localhost:8000/health` → `{"status":"ok","environment":"local"}` ; interactive docs at `http://localhost:8000/docs` . Smoke-test login:

```
curl.exe -X POST http://localhost:8000/auth/login -H "Content-Type: application/json" `
     -d '{\"email\":\"admin@example.com\",\"password\":\"ChangeMe123!\"}'
# (use curl.exe, not `curl`: in Windows PowerShell 5.1 `curl` is an alias for Invoke-WebRequest)
# → {"access_token":"eyJ…","token_type":"bearer"}
```

4. Run the Celery workers and beat

The pipeline uses three queues (processing-queues.md). Locally the simplest setup is **one worker that serves all three** (second terminal):

```
cd backend
.\.venv\Scripts\Activate.ps1
celery -A app.tasks.celery_app worker --loglevel=info --pool=threads --concurrency=8 `
       -Q extraction_queue,vision_queue,forensics_queue,celery
```

Windows cannot use the prefork pool — use `--pool=threads` (or `--pool=solo` , one task at a time). On Linux/macOS run one worker per queue exactly as production does:

```
sh start-worker.sh extraction     # threads, EXTRACTION_WORKER_CONCURRENCY (4)
sh start-worker.sh vision         # threads, VISION_WORKER_CONCURRENCY (4)
sh start-worker.sh forensics      # prefork, one process per CPU (FORENSICS_WORKER_CONCURRENCY=0)
```

or all of them in Linux containers: `docker compose --profile workers up -d` (add `--scale forensics-worker=N`). The worker lists its tasks (`process_document` , `run_document_checks` , `run_cross_document_checks` , `run_metadata_forensics` , `run_tampering_checks` , `run_duplicate_check` , `run_visual_inconsistency_review` , `run_signature_detection` , `run_signature_comparison` , `score_case` , `reconcile_usage_stats` , `log_queue_metrics` , `requeue_stuck_documents`) and then `ready` . **Without a worker, uploads succeed but every document stays `pending` forever** — the API only enqueues.

Optionally run **beat** and the worker for its jobs (per-minute queue metrics line, stuck-document recovery every 5 minutes, nightly usage reconciliation). These jobs run on their own `housekeeping_queue`, so they need this worker; run exactly one of each:

```
celery -A app.tasks.celery_app beat --loglevel=info
celery -A app.tasks.celery_app worker -Q housekeeping_queue -n housekeeping@%h --pool=solo --loglevel=info
# Linux/macOS/containers: both in one process with `sh start-beat.sh` (`--beat` does not work on Windows)
```

> **Restart the workers after every backend code change.** Unlike `uvicorn --reload`, Celery does not auto-reload; it keeps running the old code.

5. Run the frontend

```
cd frontend
npm install                # or `npm ci` for an exact lockfile install
npm run dev                # http://localhost:5173
```

No env file is needed: the dev server proxies `/api/*` → `http://127.0.0.1:8000/*` (prefix stripped), so no CORS setup is required. Sign in with `admin@example.com` / `ChangeMe123!` (platform admin) or a company account you created.

Other scripts: `npm run build` (type-check + production build into `dist/`, route-level code splitting), `npm run lint` (oxlint), `npm run preview`.

6. Try the pipeline

1. Sign in as a company `user` or reviewer → **New case** → pick a case type → drop PDFs from `sample-documents/` (e.g. the `Sample9`, `Sample11` Arabic pairs, or the PDFs inside `tampered_test_samples/tampered_test_samples.zip`; the `_Invoice` / `_Evidence` pairs share a case). Only PDFs within the company's limit (10 MB by default) are accepted; images, password-protected and corrupted PDFs are refused with a specific message.
2. Open the case; the page polls every 5 s until *Analyzing* becomes a risk tier and *Approve* unlocks.
3. Optionally, on the upload screen, **Set reference signature** to trigger signature comparison.
4. As the platform admin, watch **Platform › Processing queues** and **Billing & usage**.

Re-uploading a sample you already uploaded *in the same company* **will** flag duplicates (10). The `stitch_docauth_document_review_platform/` folder holds static HTML design mockups, not part of the app.

Tests

```
cd backend
pytest                     # ~480 tests, ~4 minutes; external services are mocked (SQLite)
```

The database-level tests need a real, migrated Postgres and are skipped otherwise:

```
$env:FDDT_RLS_TEST_DATABASE_URL = "postgresql+psycopg2://docauth:docauth@localhost:5432/docauth"
pytest tests/test_rls_postgres.py       # Row-Level Security, role privileges, upserts as the app role
$env:FDDT_PGBOUNCER_HOST = "localhost"
pytest tests/test_pgbouncer_rls.py      # tenant context never leaks through transaction pooling
```

The rate-limiter tests need Redis on `localhost:6379`. `backend/loadtest/` holds the load-test harness (`docker-compose.loadtest.yml`).

Housekeeping and troubleshooting

Symptom	Cause / fix
Documents stuck at `pending`	No worker is consuming that queue, or Redis is unreachable. Check **Platform › Processing queues**
Upload rejected with a message	Working as designed: PDF only, within the company's size limit, not empty, not corrupted, not password-protected (01)
Upload returns 503	`AZURE_STORAGE_CONNECTION_STRING` missing/invalid
Document shows `failed` with an Azure error	Check the document's `processing_error`, and the Document Intelligence / OpenAI keys, endpoint and deployment name
Visual review / signature checks fail, extraction works	The OpenAI deployment is not vision-capable (must contain `gpt-4o`, `gpt-4.1`, `gpt-4-turbo` or `gpt-4-vision`)
Azure 429s / slow vision queue	Raise the deployment's TPM, or lower `AZURE_OPENAI_MAX_*_PER_MINUTE` to ~80 % of the real quota
Case stuck at *Analyzing*	A pipeline task has not finished; `GET /cases/{id}` → `pipeline.pending` lists what it is waiting for
"permission denied for table …" in a worker log	A write needs a privilege the app role lacks — every tenant table's grants are explicit (see schema)
Login works but a company user sees nothing	They belong to another company than the data, or the company is suspended
Port 5432/6379/6432 in use	Another Postgres/Redis/PgBouncer; stop it or set `POSTGRES_PORT` / `REDIS_PORT` / `PGBOUNCER_PORT` (and the URLs)
`alembic` cannot connect	Run it from `backend/`, with the containers healthy and `DATABASE_URL` matching
Backend code change has no effect on processing	Restart the Celery workers
Wrong/empty `.env` values	Confirm you started the process from `backend/`

Useful commands:

```
docker exec docauth-postgres psql -U docauth -d docauth -c "select name, is_active from companies"
python backfill_field_locations.py [CASE-XXXXXXXX]   # re-locate extracted fields (re-runs OCR only, no LLM)
python -m scripts.calibrate_llm_tokens               # re-measure token usage after a model/prompt change
cd ..\docs\tools; ..\..\backend\.venv\Scripts\python gen_db_docs.py   # regenerate the schema docs
```